BTEUP EXAM NOTES | ENGLISH EDITION

Mobile App Development

Unit II: Application Design Essentials: Activities to Permissions

About these notes

Part A explains every topic in very simple English with real-life examples, so a beginner can understand it from zero.
Part B gives important exam questions with full 5-mark model answers, ready to write in the exam.
At the end: quick revision sheet, common mistakes, and a self-test.

Syllabus covered (Unit II)

Android Application Design Essentials: Anatomy of an Android application, Android terminologies, Application Context, Activities, Services, Intents, Receiving and Broadcasting Intents, Android Manifest File and its common settings, Using Intent Filter, Permissions.

Unit Overview and Contents

Simple idea: An Android app is not one big program. It is a **team of components**: Activities (screens), Services (background work), Broadcast Receivers (listeners) and Content Providers (data sharing). **Intents** are the messages that connect them, the **manifest** is the app's ID card that lists them, **intent filters** say what each can handle, and **permissions** decide what the app is allowed to do. This is the **most important unit** of the subject.

Topic	Exam importance	Where in notes
Anatomy and terminologies	Medium	Part A: Topic 1, 2 | Part B: Q1
Application Context	High	Part A: Topic 3 | Part B: Q2
Activities and life cycle	Very High (almost sure)	Part A: Topic 4 | Part B: Q3, Q4
Services	Very High	Part A: Topic 5 | Part B: Q5, Q6
Intents	Very High	Part A: Topic 6 | Part B: Q7
Broadcast receivers	Very High	Part A: Topic 7 | Part B: Q8
Manifest file and settings	Very High	Part A: Topic 8 | Part B: Q9
Intent filter	Very High	Part A: Topic 9 | Part B: Q10
Permissions	Very High	Part A: Topic 10 | Part B: Q11, Q12

Contents

Part A: Easy Explanation	Part B: Important Questions with Answers
Topic 1: Anatomy of an app Topic 2: Android terminologies Topic 3: Application Context Topic 4: Activities Topic 5: Services Topic 6: Intents Topic 7: Receiving and broadcasting intents Topic 8: Manifest file and common settings Topic 9: Intent filter Topic 10: Permissions	Q1 Android terminologies (Likely) Q2 Context (Likely) Q3 Activity life cycle (Very Likely) Q4 Activity code, passing data (Very Likely) Q5 Services (Very Likely) Q6 Service example (Likely) Q7 Intents (Very Likely) Q8 Broadcast receiver (Very Likely) Q9 Manifest settings (Very Likely) Q10 Intent filter (Very Likely) Q11 Permissions (Very Likely) Q12 Runtime permission code (Likely) Q13 Differences (Possible)

How to study this unit (suggested time: 1 to 2 days)

1) Read Part A once to understand. 2) Practice the **Activity life cycle diagram** until you can draw it without looking. 3) Learn the code snippets (activity, intent, service, receiver, permission). 4) Revise with the Quick Revision Sheet and attempt the Self-Test.

PART A: Easy Explanation (Simple English)

Topic 1: Anatomy of an Android Application (quick recap)

You saw the folders and files of a project in Unit I. Now look at the app as a **set of building blocks**.

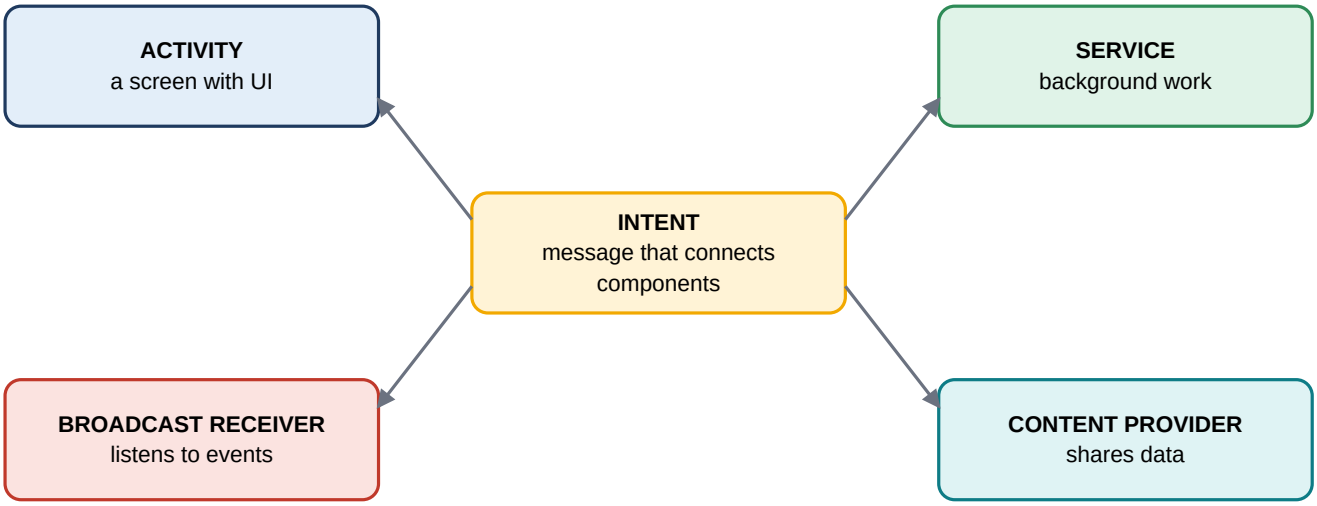


Figure 1: Four building blocks of an Android app, connected by Intents

Part	Role
Components	Activity, Service, Broadcast Receiver, Content Provider. Declared in the manifest.
Intents	Messages that start components or carry data between them.
Manifest	ID card: lists components, permissions, features.
Resources (res/)	Layouts, strings, images, colours. Reached through the R class.
Gradle files	Build settings: applicationId, minSdk, dependencies.

What happens when you tap an app icon

- The launcher sends an **Intent** with action MAIN and category LAUNCHER (taken from the manifest).
- The system creates a **new Linux process** for the app (each app runs in its own sandbox).
- The **Application** object is created first, then the launcher **Activity** is created and its life-cycle methods run (onCreate, onStart, onResume).

Topic 2: Android Terminologies

These words appear again and again. Learn them as a vocabulary list.

Term	Meaning
Activity	One screen with a user interface (login screen).
Service	A component that does work in the background without a UI.
Broadcast Receiver	Component that listens for system-wide or app messages (battery low).
Content Provider	Component that shares data with other apps (contacts).
Intent	A message used to start a component or send data.
Intent Filter	Declares which intents a component can handle.

Term	Meaning
View / ViewGroup	View = a UI element (Button, TextView). ViewGroup = a container of views (a layout).
Layout	XML file that arranges views on a screen.
Fragment	A reusable part of a screen that lives inside an activity.
Context	A handle to the app environment (resources, services, system).
Manifest	AndroidManifest.xml, the ID card of the app.
Resources / R class	Non-code files (layouts, strings, images); R gives them IDs.
Bundle	A key-value container used to pass or save small data.
Task / Back stack	The stack of activities the user has visited.
Toast	A small message that appears for a moment on the screen.
Notification	A message shown in the status bar and notification drawer.
Widget	A mini app view shown on the home screen (also used for UI elements).
APK / AAB	Installable package / publishing bundle of an app.
AVD / ADB	Virtual device (emulator) / command-line bridge to a device.
Logcat	Window that shows log messages for debugging.
Permission	Approval needed to use protected data or features.
ART / DEX	Runtime that runs the app / bytecode format of Android.

Topic 3: Application Context

Real-life example

Think of **Context** as an **ID badge with access rights**. With the badge you can open rooms (resources), call the lift (system services) and enter other offices (start activities). Without a Context, your code cannot reach anything in the Android world.

Definition

Context is an abstract class that provides an interface to **global information about the application environment**. It lets code access resources, assets, databases, system services, and launch activities and services or send broadcasts.

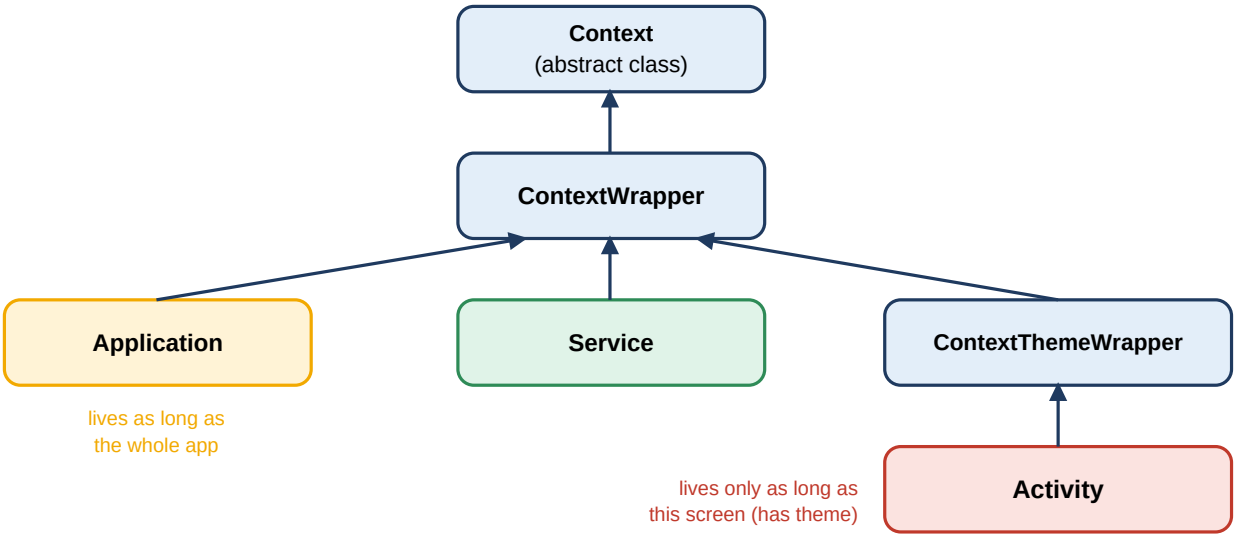


Figure 2: Context class hierarchy

Type of context	How to get it	Lifetime
Application context	getApplicationContext()	Whole app (until the process ends)
Activity context	this or MainActivity.this	Only while that activity exists
Service context	this (inside a Service)	While the service exists
Base context	getBaseContext()	The context wrapped by a ContextWrapper

Which context for which job?

Operation	Activity context	Application context
Show a dialog	Yes	No (crashes)
Inflate a themed layout	Yes	Works, but uses default theme
Start an activity	Yes	Only with FLAG_ACTIVITY_NEW_TASK
Start a service / send a broadcast	Yes	Yes
Load a resource / use system service	Yes	Yes
Store in a long-lived object (singleton)	Risk of memory leak	Safe

- **Memory leak:** if a long-lived object holds an Activity context, the activity cannot be cleared from memory after it is closed. Use the **application context** for long-lived objects.
- **Application class:** `android.app.Application` keeps global state of the app. It is created **before any activity, service or receiver**. Register a custom one with `android:name=".MyApp"` in the manifest.

Context examples

```
// Using the activity context (this) and the application context
Toast.makeText(this, "Hello", Toast.LENGTH_SHORT).show();
Context appContext = getApplicationContext();

// A custom Application class
public class MyApp extends Application {
    @Override
    public void onCreate() {
        super.onCreate();   // runs once when the app process starts
    }
}
```

Topic 4: Activities

Real-life example

An activity is **one page of a book**. A book (app) has many pages; only one page is open at a time. The reader can turn forward (new activity) or go back (Back button).

Definition

An **Activity** is an application component that provides **one screen** with which users can interact, such as a login screen or a list. Every activity must be **declared in the manifest**.

- An activity usually extends **AppCompatActivity**, and its screen design comes from an XML layout set with `setContentView()`.
- The system manages activities with callback methods called the **life cycle**. The system, not you, decides when they run.

Activity life cycle

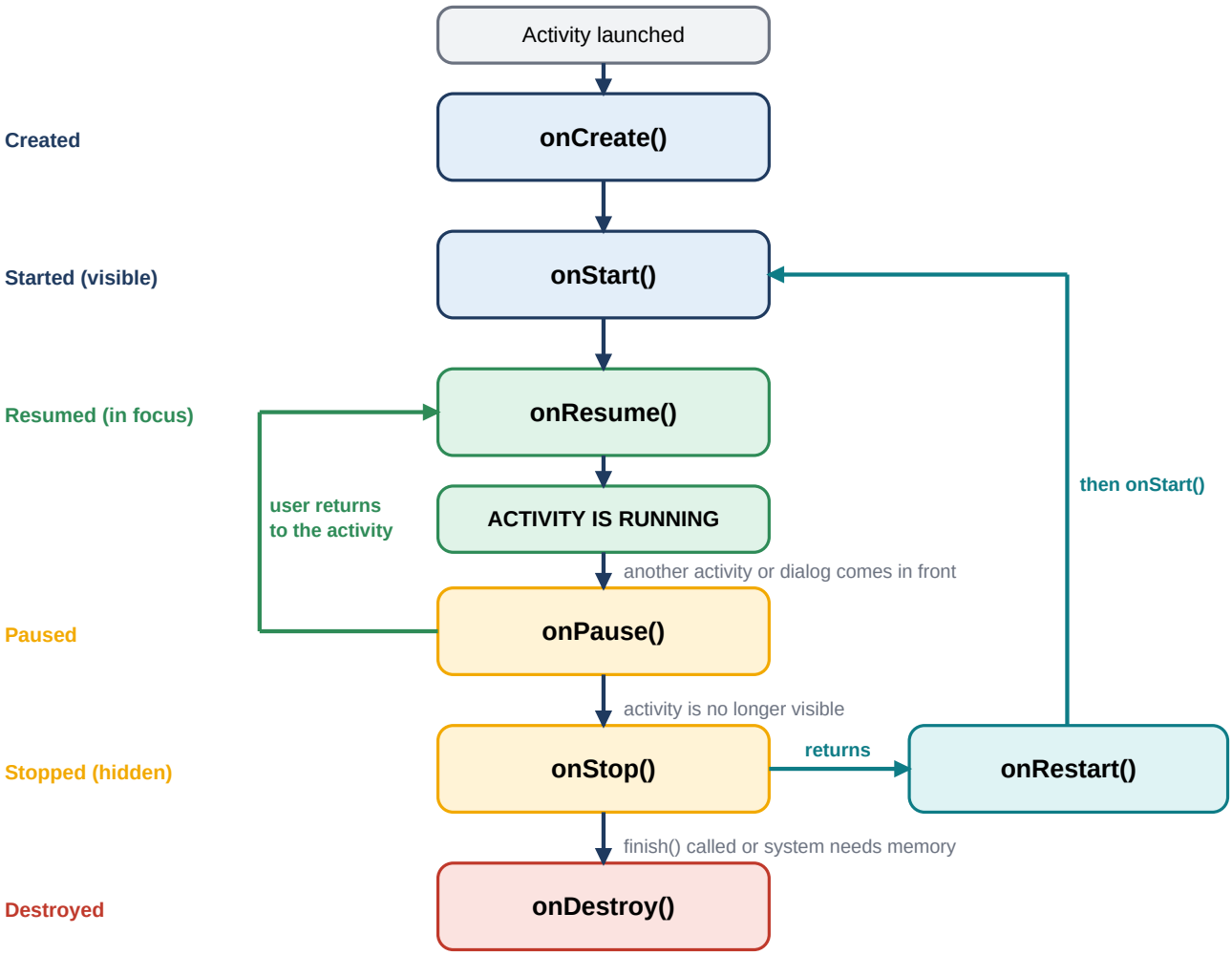


Figure 3: Activity life cycle

Method	When it is called
onCreate()	Activity is created (once). Always followed by onStart().
onStart()	Activity becomes **visible**.
onResume()	Activity is in the foreground and the user can interact.
onPause()	Another activity or dialog is coming in front; activity may be partly visible.
onStop()	Activity is **no longer visible**.
onRestart()	Activity comes back from the stopped state; followed by onStart().
onDestroy()	Activity is being destroyed (finish() called or system needs memory).

Do this here	Method
Set the layout, find views, initialise data (once)	onCreate()
Start animations, register things that need a visible UI	onStart() / onResume()
Pause animations, save light data, release camera/sensors	onPause()
Save data permanently, stop heavy work	onStop()
Final cleanup (threads, listeners)	onDestroy()

What is called when? (common scenarios)

Scenario	Methods called, in order
App opens for the first time	onCreate, onStart, onResume
User presses the Home button	onPause, onStop
User returns to the app	onRestart, onStart, onResume
User presses the Back button (activity closes)	onPause, onStop, onDestroy
A dialog appears in front, then closes	onPause, then onResume
Screen is rotated (configuration change)	onPause, onStop, onDestroy, then onCreate, onStart, onResume (activity is recreated)
Activity A starts Activity B	A.onPause, B.onCreate, B.onStart, B.onResume, A.onStop

- **Three lifetimes: entire lifetime** = onCreate to onDestroy; **visible lifetime** = onStart to onStop; **foreground lifetime** = onResume to onPause.
- **Four states:** Resumed (running), Paused (partly visible), Stopped (hidden but in memory), Destroyed.
- **Process death:** a stopped activity can be killed when memory is low, so **save important data**.
- Always call the **super** method first (for example `super.onCreate(savedInstanceState)`).

MainActivity.java: logging every life-cycle method (watch in Logcat)

```
public class MainActivity extends AppCompatActivity {
    private static final String TAG = "Lifecycle";

    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Log.d(TAG, "onCreate");
    }
    @Override protected void onStart()   { super.onStart();   Log.d(TAG, "onStart"); }
    @Override protected void onResume()  { super.onResume();  Log.d(TAG, "onResume"); }
    @Override protected void onPause()   { super.onPause();   Log.d(TAG, "onPause"); }
    @Override protected void onStop()    { super.onStop();    Log.d(TAG, "onStop"); }
    @Override protected void onRestart() { super.onRestart(); Log.d(TAG, "onRestart"); }
    @Override protected void onDestroy() { super.onDestroy(); Log.d(TAG, "onDestroy"); }
}
```

Saving and restoring state

When the screen rotates, the activity is destroyed and recreated, so variables are lost. Save small data in a **Bundle** in `onSaveInstanceState()` and read it back in `onCreate()`.

Saving state across rotation

```
private int count = 0;

@Override
protected void onSaveInstanceState(Bundle outState) {
    super.onSaveInstanceState(outState);
    outState.putInt("count", count);          // save
}

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    if (savedInstanceState != null) {
        count = savedInstanceState.getInt("count");   // restore
    }
}
```

Starting another activity and passing data

Two activities

```
// In MainActivity: start SecondActivity and send data
Button btn = findViewById(R.id.btnNext);
btn.setOnClickListener(v -> {
    Intent intent = new Intent(MainActivity.this, SecondActivity.class);
    intent.putExtra("name", "Amrit");
    startActivity(intent);
});

// In SecondActivity: receive the data
String name = getIntent().getStringExtra("name");
TextView tv = findViewById(R.id.tvName);
tv.setText("Hello " + name);

// Manifest: every activity must be declared
// <activity android:name=".SecondActivity" android:exported="false" />
```

- **finish()** closes the current activity. The user returns to the previous one.
- **Back stack:** each new activity is pushed on a stack; the Back button pops the top one.

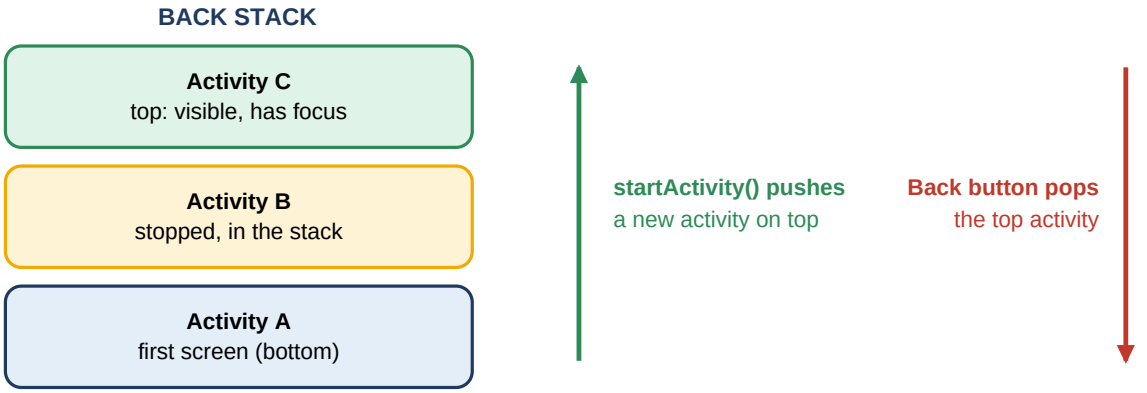


Figure 4: Back stack of activities

Launch modes (how a new instance is created)

Mode (android:launchMode)	Behaviour
standard (default)	A new instance is created every time, even if one already exists.
singleTop	If the activity is already at the top of the stack, it is reused (onNewIntent()). Otherwise a new one is created.
singleTask	Only one instance in the task; activities above it are cleared.
singleInstance	Like singleTask, but the activity is alone in its own task.

Topic 5: Services

Real-life example

A service is like the **kitchen staff** of a restaurant: they work behind the scenes while customers (the user) see only the dining hall (the screen). Music playing after you leave the music screen is a service at work.

Definition

A **Service** is an application component that can perform **long-running operations in the background** and does **not provide a user interface**.

Types of services

Type	Meaning	Example
Foreground	Does work the user notices; must show a **notification**. Rarely killed.	Music player, fitness tracker
Background	Work the user does not notice. Heavily restricted since Android 8.	Compacting storage
Bound	Other components bind to it with **bindService()**; works like a client-server; runs only while clients are bound.	Music service controlled by an activity

- **Started service:** begun with `startService()`; keeps running until it calls **stopSelf()** or another component calls **stopService()**.
- A service runs on the **main (UI) thread** by default. Do heavy work on a separate thread, otherwise the app may show **ANR** (Application Not Responding).
- A service can be both started and bound.

Service life cycle

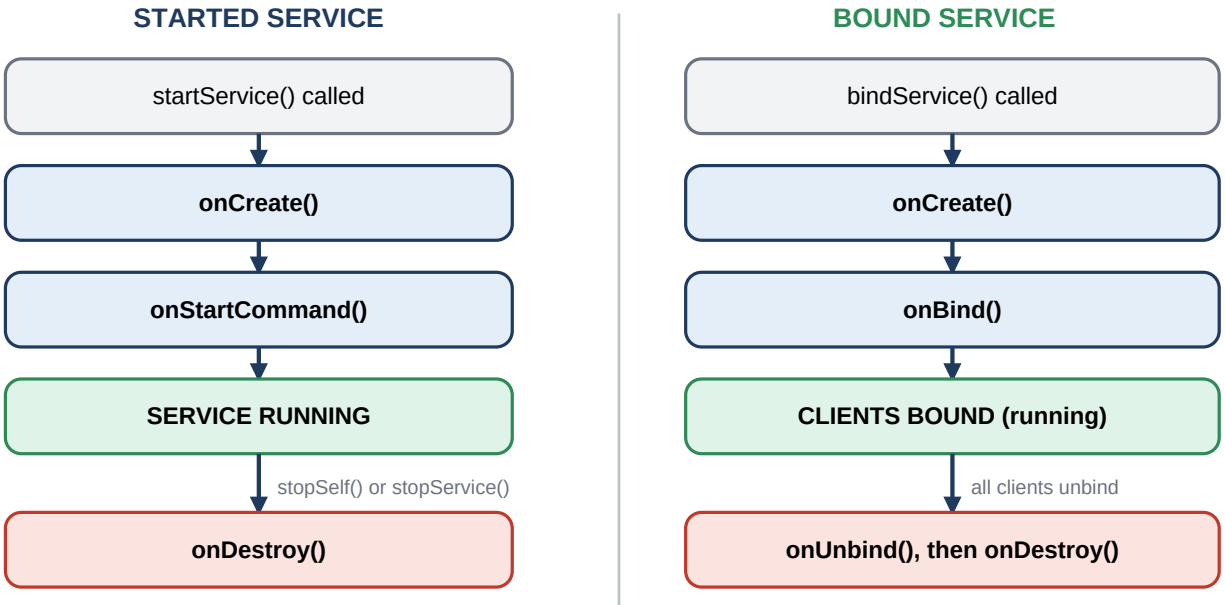


Figure 5: Life cycle of started and bound services

Callback	Purpose
onCreate()	Called once when the service is first created (one-time setup).
onStartCommand()	Called each time startService() is called; does the work. Returns START_STICKY, START_NOT_STICKY or START_REDELIVER_INTENT.
onBind()	Called by bindService(). Returns an IBinder; return null if binding is not allowed.

Callback	Purpose
onUnbind()	Called when all clients have unbound.
onDestroy()	Called when the service is no longer used; clean up here.

MusicService.java (started service)

```
public class MusicService extends Service {
    private MediaPlayer player;

    @Override
    public void onCreate() {
        super.onCreate();
        player = MediaPlayer.create(this, R.raw.song);   // song.mp3 in res/raw
        player.setLooping(true);
    }

    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        player.start();
        return START_STICKY;          // restart the service if the system kills it
    }

    @Override
    public IBinder onBind(Intent intent) {
        return null;                  // not a bound service
    }

    @Override
    public void onDestroy() {
        super.onDestroy();
        player.stop();
        player.release();
    }
}
```

Starting, stopping and declaring the service

```
// Start and stop from an activity (for example in button click listeners)
startService(new Intent(this, MusicService.class));
stopService(new Intent(this, MusicService.class));

// Declare the service in the manifest (inside <application>)
// <service android:name=".MusicService" android:exported="false" />
```

Modern rules you should know

1) Since Android 8.0 (API 26), an app in the background cannot freely start background services. Use **WorkManager** for background work, or a **foreground service** (with a notification) for ongoing tasks the user knows about. **2) IntentService is deprecated**; use WorkManager. **3)** Always use **explicit intents** to start or bind a service (an implicit intent throws an exception). **4)** On Android 14 (API 34), a foreground service must declare a **foreground service type**.

Bound service (outline)

```
// Bound service (outline)
public class LocalService extends Service {
    private final IBinder binder = new LocalBinder();
    public class LocalBinder extends Binder {
        LocalService getService() { return LocalService.this; }
    }
    @Override public IBinder onBind(Intent intent) { return binder; }
    public int getRandomNumber() { return new Random().nextInt(100); }
}

// In the activity
bindService(new Intent(this, LocalService.class), connection, Context.BIND_AUTO_CREATE);
// in onStop():  unbindService(connection);
```

Topic 6: Intents

Real-life example

An intent is a **request slip**. You can write 'Please give this slip to Ramesh' (explicit) or 'Whoever can repair a phone, please help' (implicit). The system then finds the right person.

Definition

An **Intent** is a **messaging object** used to request an action from another app component. It is used to **start an activity**, **start a service** and **deliver a broadcast**.

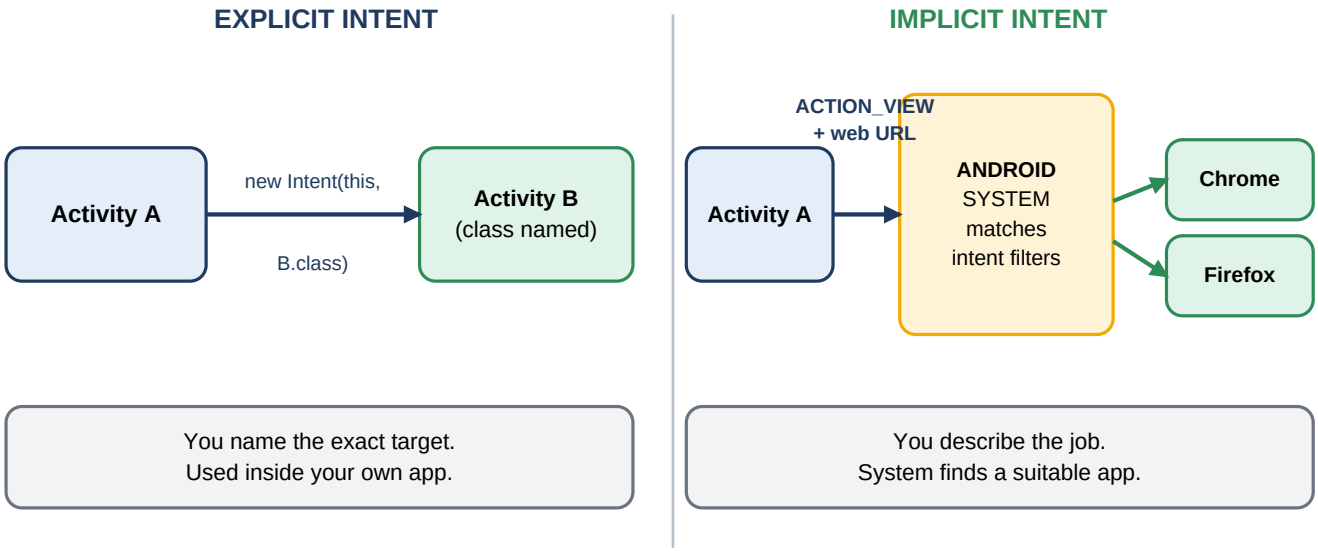


Figure 6: Explicit and implicit intents

Point	Explicit intent	Implicit intent
Target	Names the exact component class	Describes an action; no class named
Used for	Moving inside your own app	Using other apps (browser, dialer, share)
Who decides	You	Android system (matches intent filters)
Example	new Intent(this, SecondActivity.class)	new Intent(Intent.ACTION_VIEW, uri)

Parts of an Intent

Part	Meaning
Component name	Target class (explicit intents only)
Action	What to do: ACTION_VIEW, ACTION_SEND, ACTION_DIAL, ACTION_MAIN

Part	Meaning
Data	URI of the data (tel:100, https://..., content://...) and its MIME type
Category	Extra information about the kind of component: CATEGORY_LAUNCHER, CATEGORY_BROWSABLE
Extras	Key-value pairs carrying data (putExtra / getStringExtra)
Flags	Instructions about how to launch, e.g. FLAG_ACTIVITY_NEW_TASK

Intent examples

```
// 1. Explicit intent: open your own SecondActivity
Intent i = new Intent(MainActivity.this, SecondActivity.class);
startActivity(i);

// 2. Implicit: open a web page
Intent web = new Intent(Intent.ACTION_VIEW, Uri.parse("https://www.example.com"));
startActivity(web);

// 3. Implicit: open the dialer with a number (no permission needed)
Intent dial = new Intent(Intent.ACTION_DIAL, Uri.parse("tel:100"));
startActivity(dial);

// 4. Implicit: share text with any app
Intent send = new Intent(Intent.ACTION_SEND);
send.setType("text/plain");
send.putExtra(Intent.EXTRA_TEXT, "Hello from my app");
startActivity(Intent.createChooser(send, "Share via"));
```

- **ACTION_DIAL** only opens the dialer (no permission). **ACTION_CALL** places the call directly and needs the **CALL_PHONE** permission.
- If no app can handle an implicit intent, the app crashes with **ActivityNotFoundException**. Check first with `intent.resolveActivity(getPackageManager())` (on Android 11+ you may need a `<queries>` entry).

Getting a result back from an activity

Result from another activity

```
// Modern way (Activity Result API) in MainActivity
ActivityResultLauncher<Intent> launcher = registerForActivityResult(
    new ActivityResultContracts.StartActivityForResult(),
    result -> {
        if (result.getResultCode() == RESULT_OK && result.getData() != null) {
            String answer = result.getData().getStringExtra("answer");
        }
    });
launcher.launch(new Intent(this, SecondActivity.class));

// In SecondActivity: send the result back
Intent data = new Intent();
data.putExtra("answer", "42");
setResult(RESULT_OK, data);
finish();
```

Old way in textbooks

Older books use `startActivityForResult(intent, 1)` and override `onActivityResult(int requestCode, int resultCode, Intent data)`. It still works but is **deprecated** in favour of the Activity Result API shown above. Learn both for the exam.

Topic 7: Receiving and Broadcasting Intents

Real-life example

A broadcast is a **public announcement on a loudspeaker** at a railway station: 'Train 12345 is late'. Everyone who is listening (every registered receiver) hears it, and each decides what to do.

Definition

A **Broadcast Receiver** is a component that **listens for broadcast intents** sent by the system or by apps, and reacts to them. A **broadcast** is a message sent to **all interested receivers**.

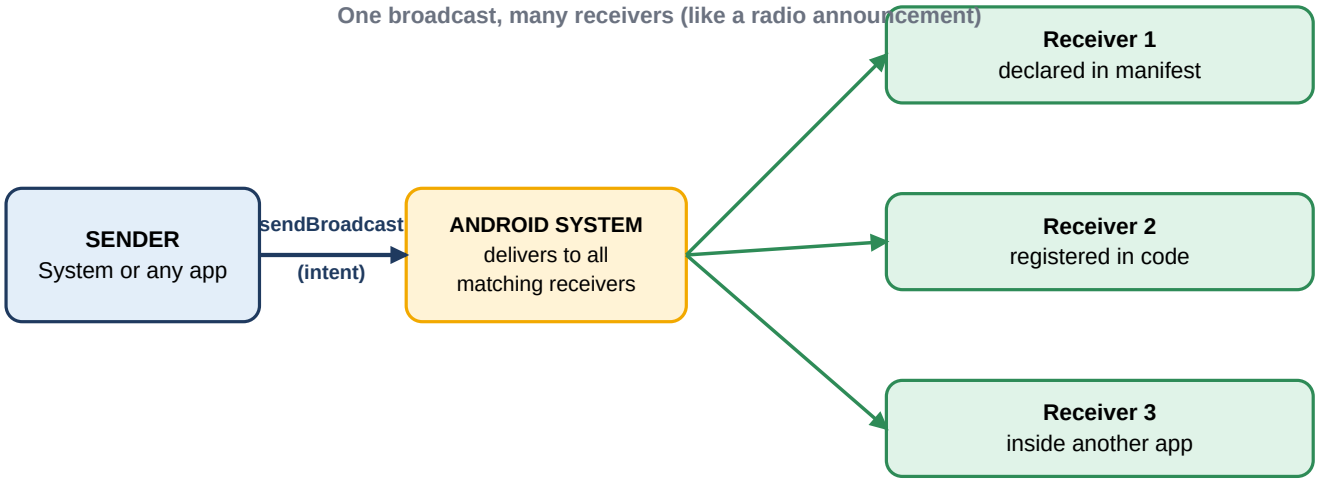


Figure 7: Broadcasting an intent to many receivers

Common system broadcasts

Action	Sent when
ACTION_BOOT_COMPLETED	The device finishes booting (needs RECEIVE_BOOT_COMPLETED permission)
ACTION_BATTERY_LOW / ACTION_BATTERY_OKAY	Battery becomes low / okay
ACTION_POWER_CONNECTED / DISCONNECTED	Charger is plugged in / removed
ACTION_AIRPLANE_MODE_CHANGED	Airplane mode is switched
ACTION_SCREEN_ON / SCREEN_OFF	Screen turns on / off (only context-registered receivers get it)
SMS_RECEIVED (Telephony)	An SMS arrives (restricted; needs special permission)

Two ways to register a receiver

Point	Manifest-declared (static)	Context-registered (dynamic)
Where	<receiver> tag in AndroidManifest.xml	registerReceiver() in code
Works when	Even if the app is not running (system starts it)	Only while the registering activity/context is alive
Restriction	Since Android 8.0 most implicit broadcasts cannot be declared here (a few exceptions such as BOOT_COMPLETED)	Not restricted; must unregister when done

MyReceiver.java

```
// 1. A receiver class
public class MyReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        Toast.makeText(context, "Broadcast received: " + intent.getAction(),
                       Toast.LENGTH_SHORT).show();
    }
}
```

Registering a receiver (static and dynamic)

```
// 2a. Register in the manifest (static), e.g. to run after boot
// <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />
// <receiver android:name=".MyReceiver" android:exported="true">
//     <intent-filter>
//         <action android:name="android.intent.action.BOOT_COMPLETED" />
//     </intent-filter>
// </receiver>

// 2b. Register in code (dynamic) inside an activity
private final MyReceiver receiver = new MyReceiver();

@Override protected void onStart() {
    super.onStart();
    IntentFilter filter = new IntentFilter(Intent.ACTION_BATTERY_LOW);
    ContextCompat.registerReceiver(this, receiver, filter,
            ContextCompat.RECEIVER_EXPORTED);       // listening to system broadcasts
}
@Override protected void onStop() {
    super.onStop();
    unregisterReceiver(receiver);                   // always unregister
}
```

Sending a broadcast

```
// 3. Broadcasting your own intent (inside your own app)
Intent intent = new Intent("com.example.myapp.MY_ACTION");
intent.setPackage(getPackageName());     // keep it inside this app
intent.putExtra("msg", "Hello receivers");
sendBroadcast(intent);

// sendOrderedBroadcast(intent, null) delivers to one receiver at a time, by priority.
```

- **onReceive()** runs on the main thread and must finish quickly (about 10 seconds). Do not do long work there; start a service or WorkManager job instead.
- **Normal broadcast** (sendBroadcast) goes to all receivers at the same time. **Ordered broadcast** (sendOrderedBroadcast) goes to one receiver at a time in priority order, and a receiver can pass on or abort it.
- **Android 14 (API 34):** if you target it and register a receiver in code for non-system broadcasts, you must specify **RECEIVER_EXPORTED** or **RECEIVER_NOT_EXPORTED**. Use ContextCompat.registerReceiver() so the flag works on older versions too.
- **Security:** a broadcast can be read by other apps. Never put secrets in it; restrict it with `setPackage()` or a permission.

Topic 8: Android Manifest File and its Common Settings

You met the manifest in Unit I. Here are the **settings you will use most often**. Every activity, service, receiver and provider must be declared here, and the manifest also holds permissions and app-wide settings.

Element	Common settings (attributes)
<uses-permission>	android:name = the permission, e.g. android.permission.INTERNET, CAMERA
<uses-feature>	android:name (e.g. android.hardware.camera), **android:required** (true or false)
<uses-sdk>	minSdkVersion, targetSdkVersion (older projects; now set in build.gradle)
<supports-screens>	Which screen sizes the app supports (small, normal, large)
<application>	**icon, roundIcon, label, theme**, allowBackup, supportsRtl, name (custom Application class), usesCleartextTraffic (allow plain HTTP)
<activity>	**name, label, icon, theme, exported, launchMode, screenOrientation** (portrait / landscape), configChanges, windowSoftInputMode, parentActivityName
<service>	name, exported, enabled, permission, foregroundServiceType, process
<receiver>	name, exported, enabled, permission, plus an <intent-filter>
<provider>	name, **authorities**, exported, grantUriPermissions
<intent-filter>	Child of a component: <action>, <category>, <data>
<permission>	Defines a custom permission: name, protectionLevel
<meta-data>	Extra name-value data for a component (for example an API key)
<queries>	Declares other apps your app needs to see (Android 11+ package visibility)

A manifest with the most common settings

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myapp">

    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.CAMERA" />
    <uses-feature android:name="android.hardware.camera" android:required="false" />

    <application
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/Theme.MyApp">

        <activity android:name=".MainActivity" android:exported="true"
            android:screenOrientation="portrait">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <activity android:name=".SecondActivity" android:exported="false"
            android:parentActivityName=".MainActivity" />

        <service android:name=".MusicService" android:exported="false" />

        <receiver android:name=".MyReceiver" android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.BOOT_COMPLETED" />
            </intent-filter>
        </receiver>

    </application>
</manifest>
```

- **Only one activity** has the MAIN + LAUNCHER filter: the one that opens when you tap the app icon.
- **android:exported:** must be set explicitly for any component with an intent filter when you target Android 12 (API 31) or higher. Use **false** unless other apps must reach the component.
- **screenOrientation:** locks the screen to portrait or landscape. **configChanges:** tells Android that the activity handles some changes (like rotation) itself, so it is not recreated.
- **usesCleartextTraffic:** apps targeting Android 9 (API 28) or higher block plain HTTP by default; use HTTPS, or allow cleartext only for testing.

Topic 9: Using Intent Filter

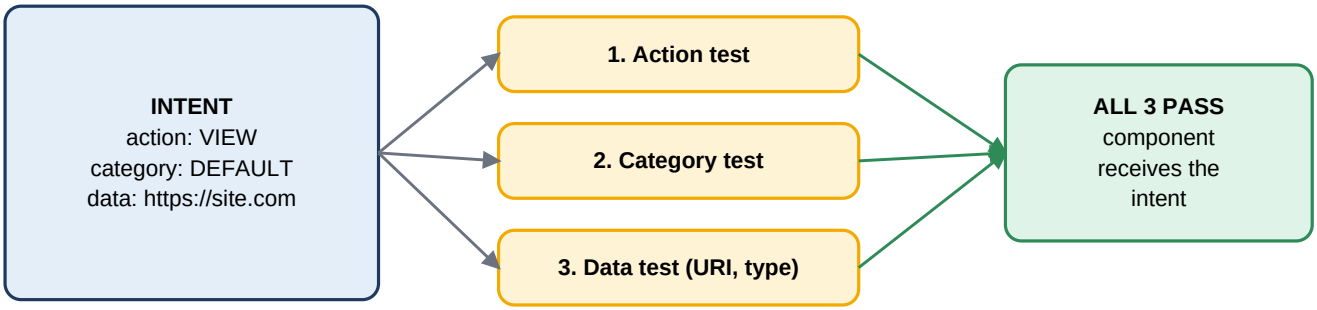
Real-life example

An intent filter is like the **board outside a shop**: 'We repair mobiles'. People looking for that service find the shop. In the same way, an intent filter tells Android what kinds of requests a component can handle.

Definition

An **intent filter** is an expression in the manifest that **declares the types of intents a component can receive**. The system compares an implicit intent with the filters of all apps and picks the matching components.

Element	Meaning	Example
<action>	The action the component can perform	android.intent.action.VIEW
<category>	The kind of component or how it is used	DEFAULT, LAUNCHER, BROWSABLE
<data>	The type of data: scheme, host, port, path, mimeType	scheme=https, host=www.example.com



The system compares the intent with each component's intent filter

Figure 8: Matching an intent against an intent filter

Matching rules

- An intent must pass **all three tests**: action, category and data.
- **Action test:** the intent's action must match one of the actions listed in the filter.
- **Category test:** every category in the intent must be in the filter. For implicit intents sent with startActivity(), the system adds **CATEGORY_DEFAULT**, so an activity that wants implicit intents must include it.
- **Data test:** the intent's URI and MIME type must match the filter's data element.
- If several apps match, the user sees a **chooser** dialog.

Common intent filters

```
<!-- 1. Launcher activity: the first screen when the app icon is tapped -->
<intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
</intent-filter>

<!-- 2. Open web links of your site in your app (deep link) -->
<intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <category android:name="android.intent.category.BROWSABLE" />
    <data android:scheme="https" android:host="www.example.com" />
</intent-filter>

<!-- 3. Receive text shared from other apps -->
<intent-filter>
    <action android:name="android.intent.action.SEND" />
    <category android:name="android.intent.category.DEFAULT" />
    <data android:mimeType="text/plain" />
</intent-filter>
```

- **Services:** always start or bind with an **explicit** intent. An implicit intent for bindService() throws an exception, and for security a service should not depend on an intent filter.
- **Broadcast receivers** use intent filters to say which broadcasts they want (static receivers in the manifest, dynamic ones with an IntentFilter object).
- **Security:** an intent filter is not a security boundary. Do not trust data received through it; validate it.

Topic 10: Permissions

Real-life example

Android is like a **secured office building**. Apps are visitors. To enter the server room (camera, contacts, location), a visitor needs a **pass**. Some passes are given at the gate (normal), but for sensitive rooms the building asks the owner (the user) to approve.

Definition

A **permission** is a **restriction that protects** sensitive data (contacts, SMS) and features (camera, location). An app must **declare** the permissions it needs in the manifest, and for dangerous ones also **ask the user at run time**.

Types of permissions (protection levels)

Type	How it is granted	Examples
Normal	Low risk. Granted **automatically at install** when declared in the manifest.	INTERNET, ACCESS_NETWORK_STATE, VIBRATE, SET_ALARM
Dangerous (runtime)	Access private data or sensitive features. The app must **ask the user at run time** (Android 6.0, API 23 and above).	CAMERA, RECORD_AUDIO, READ_CONTACTS, ACCESS_FINE_LOCATION, SEND_SMS, CALL_PHONE, POST_NOTIFICATIONS (Android 13)
Signature	Granted only to apps signed with the same certificate as the app that defined it.	Custom permissions between your own apps
Special	User grants it from a special Settings screen, not from the normal dialog.	SYSTEM_ALERT_WINDOW, WRITE_SETTINGS, MANAGE_EXTERNAL_STORAGE

- **Before Android 6.0** all permissions were shown at install time and accepted together. **From Android 6.0**, dangerous permissions are asked when the feature is used, and the user can **revoke** them later in Settings.
- Dangerous permissions are organised into **permission groups** such as CONTACTS, LOCATION, CAMERA, MICROPHONE, PHONE and SMS.
- **Android 13 (API 33):** showing notifications needs the runtime permission **POST_NOTIFICATIONS**, and media access uses READ_MEDIA_IMAGES / READ_MEDIA_VIDEO / READ_MEDIA_AUDIO instead of READ_EXTERNAL_STORAGE.

Requesting a dangerous permission at run time (steps)

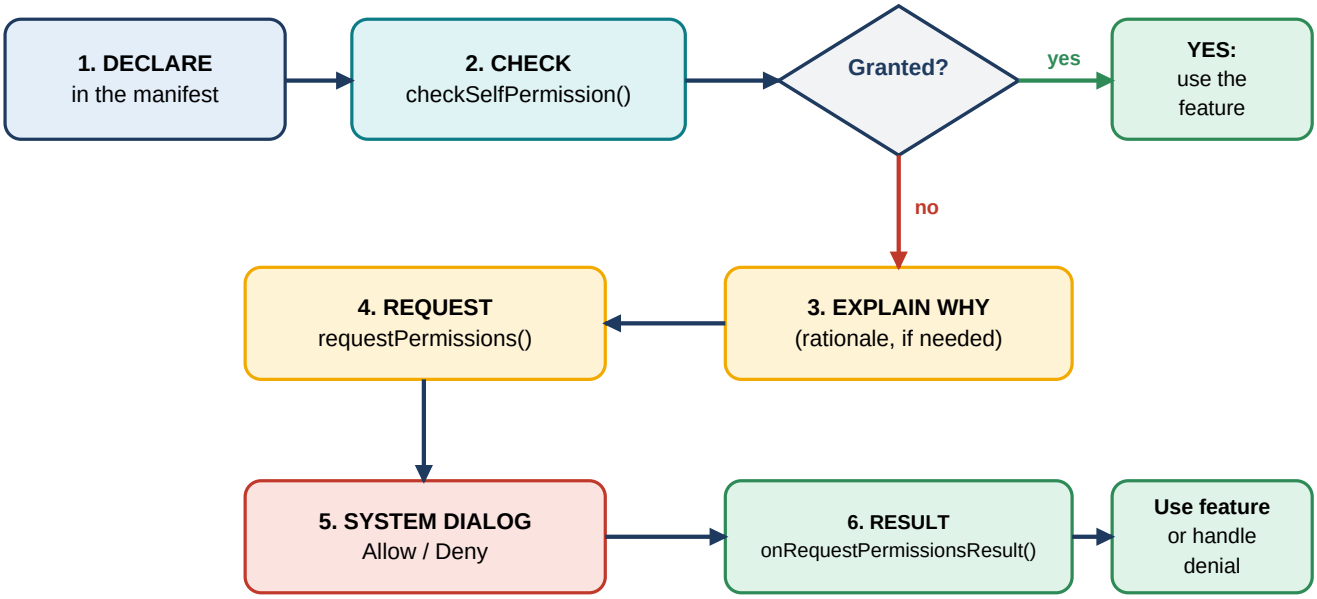


Figure 9: Runtime permission flow

Runtime permission: classic way

```
// Step 1: declare in AndroidManifest.xml
// <uses-permission android:name="android.permission.CAMERA" />

// Step 2 and 3: check, and request if not granted
private static final int CAMERA_CODE = 100;

private void startCameraFeature() {
    if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA)
            != PackageManager.PERMISSION_GRANTED) {
        ActivityCompat.requestPermissions(this,
                new String[]{Manifest.permission.CAMERA}, CAMERA_CODE);
    } else {
        openCamera();                          // already granted
    }
}

// Step 4: handle the user's answer
@Override
public void onRequestPermissionsResult(int requestCode,
        @NonNull String[] permissions, @NonNull int[] grantResults) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);
    if (requestCode == CAMERA_CODE) {
        if (grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
            openCamera();
        } else {
            Toast.makeText(this, "Camera permission denied", Toast.LENGTH_SHORT).show();
        }
    }
}
```

Runtime permission: modern way

```
// Modern way: Activity Result API (a launcher replaces onRequestPermissionsResult)
private final ActivityResultLauncher<String> cameraPermission =
    registerForActivityResult(new ActivityResultContracts.RequestPermission(),
        isGranted -> {
            if (isGranted) openCamera();
            else Toast.makeText(this, "Camera permission denied", Toast.LENGTH_SHORT).show();
        });

// when the user taps a button
cameraPermission.launch(Manifest.permission.CAMERA);
```

- **shouldShowRequestPermissionRationale()** returns true if the user denied before; explain why you need the permission before asking again.
- If the user denies and selects 'Don't ask again' (or denies repeatedly), the dialog no longer appears; guide the user to **Settings**.
- **Best practice:** ask only for what you need, ask **when needed** (not at app start), explain why, and make the app work (with fewer features) if denied.

Custom permissions

Defining and using a custom permission

```
<!-- Define a permission in your app -->
<permission android:name="com.example.myapp.permission.MY_PERM"
            android:protectionLevel="signature" />

<!-- Protect a component with it: only apps holding the permission can start it -->
<service android:name=".MyService" android:exported="true"
         android:permission="com.example.myapp.permission.MY_PERM" />
```

PART B: Important Questions with Model Answers (English)

Priority: **Very Likely** = appears most often in BTEUP. **Likely** = good chance. **Possible** = can appear. Each answer follows the exam format: **Definition, Points, Diagram/Table/Code, Conclusion**.

Q1. Explain the important Android terminologies. LIKELY

Definition / Introduction: Android has a set of basic terms that describe the building blocks and tools of an app. Knowing them is necessary to understand Android development.

Points

- **Activity:** one screen with a user interface. **Service:** background work with no UI.
- **Broadcast Receiver:** listens for system or app messages. **Content Provider:** shares data between apps.
- **Intent:** a message that starts a component or carries data. **Intent Filter:** declares which intents a component can handle.
- **View / ViewGroup / Layout:** UI element, container of views, and the XML that arranges them. **Fragment:** reusable part of a screen.
- **Context:** handle to the app environment. **Manifest:** ID card of the app. **Resources / R class:** non-code files and their IDs.
- **APK / AAB, AVD, ADB, Logcat, Gradle, Bundle, Toast, Notification, Permission, Back stack** are other common terms.

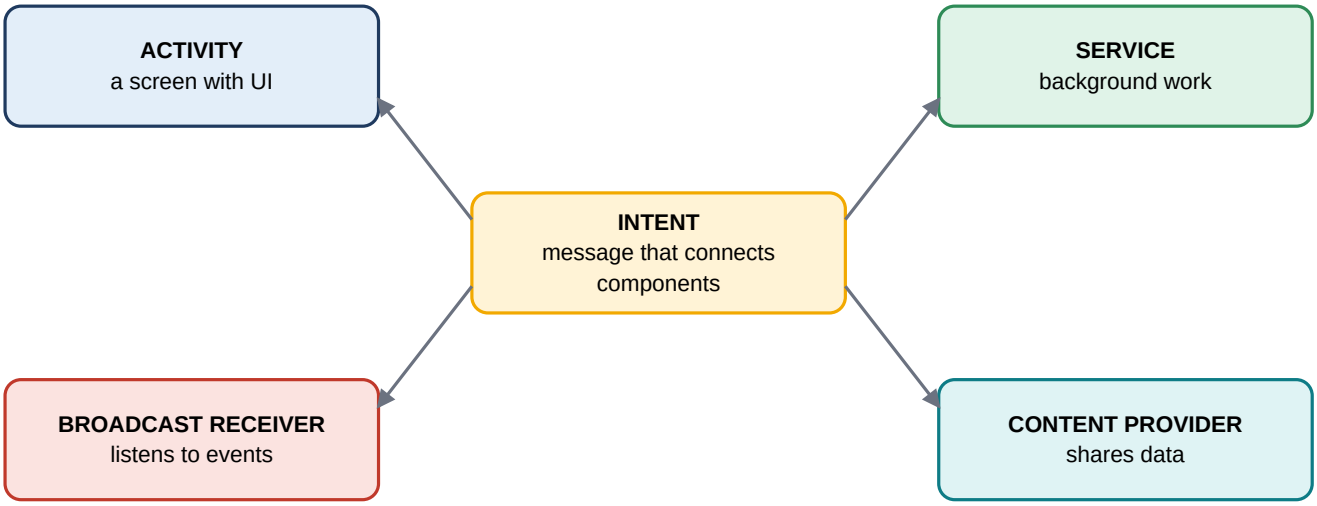


Figure: Building blocks of an Android app

Conclusion: These terms form the vocabulary of Android development and appear throughout the syllabus.

Q2. What is Context in Android? Explain Application Context and Activity Context.

LIKELY

Definition / Introduction: Context is an abstract class that provides access to global information about the application environment, such as resources, assets, databases and system services, and lets code launch components.

Points

- **Application context** is obtained with `getApplicationContext()`. It is tied to the life of the whole application and is safe to keep in long-lived objects.
- **Activity context** is obtained with `this` or `ActivityName.this`. It lives only as long as that activity and carries its theme. Dialogs and themed layouts need it.
- **Memory leak:** keeping an activity context in a long-lived object stops the activity from being freed.
- **Application class:** `android.app.Application` stores global state; it is created before any other component.

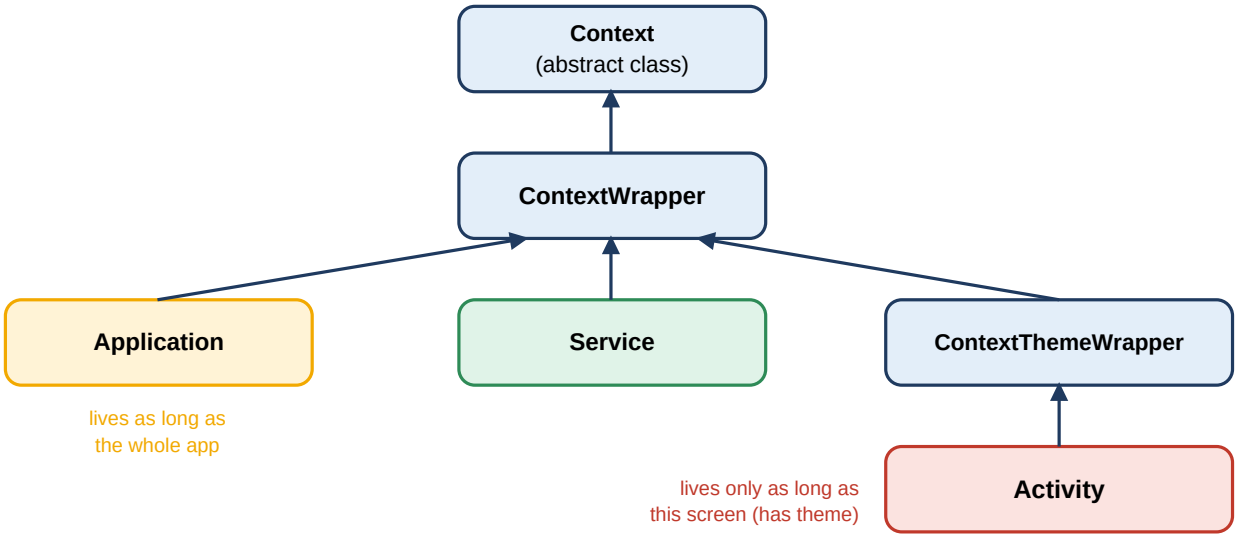


Figure: Context hierarchy

Point	Application context	Activity context
Lifetime	Whole app	That activity only
Show dialog	No	Yes
Safe in singleton	Yes	No (leak risk)

Conclusion: Choosing the right context avoids crashes and memory leaks.

Q3. What is an Activity? Explain the Activity life cycle with a diagram. VERY LIKELY

Definition / Introduction: An Activity is an application component that provides one screen with which users interact. The system manages every activity through life-cycle callback methods that run as the activity is created, shown, hidden and destroyed.

Points

- **onCreate():** called when the activity is first created; set the layout and initialise data.
- **onStart():** the activity becomes visible. **onResume():** it is in the foreground and the user can interact (running state).
- **onPause():** another activity is coming to the front; save light data and release resources.
- **onStop():** the activity is no longer visible. **onRestart():** it returns from the stopped state, followed by onStart().
- **onDestroy():** final cleanup before the activity is destroyed (finish() or system kills it).
- **Lifetimes:** entire (onCreate to onDestroy), visible (onStart to onStop), foreground (onResume to onPause).

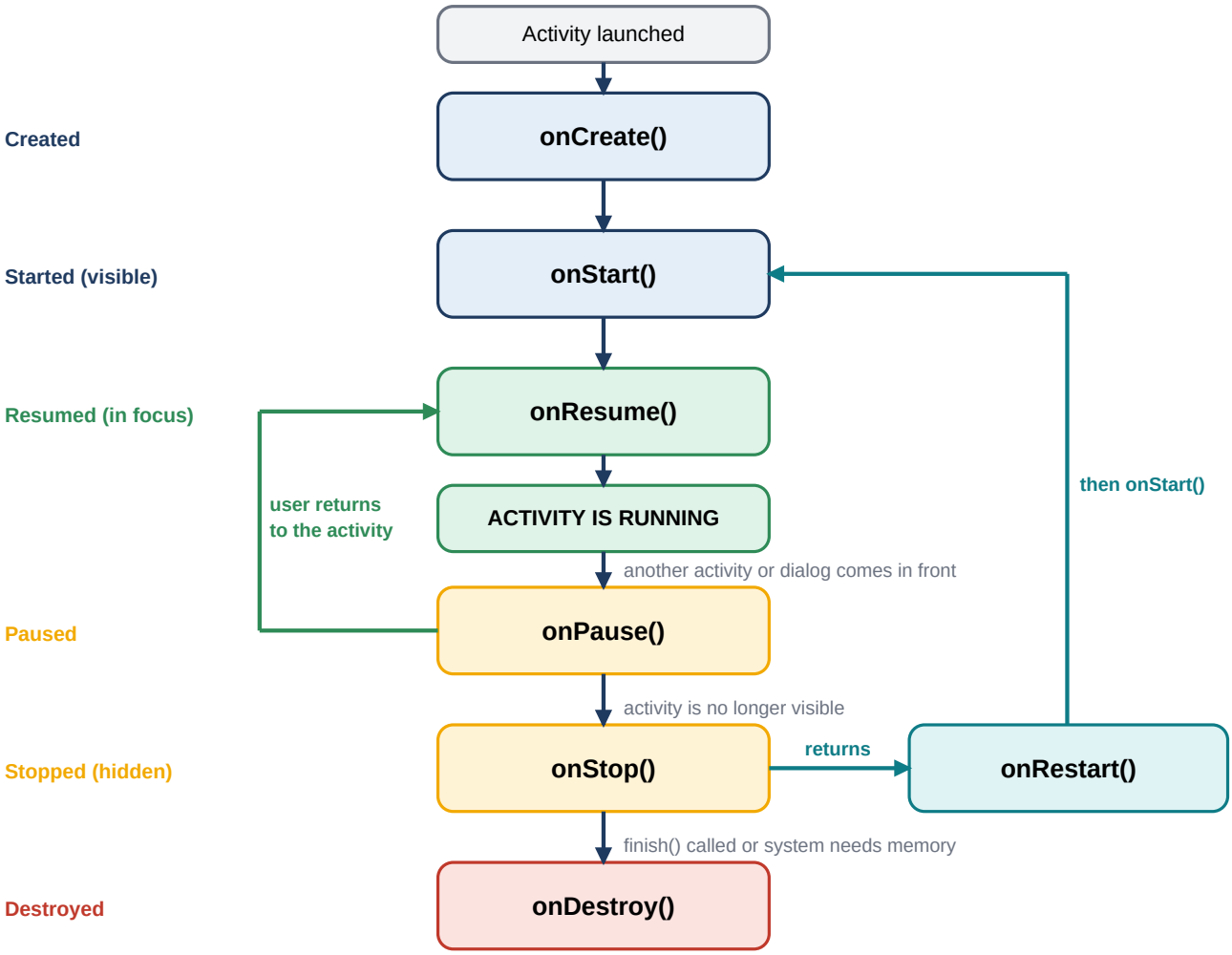


Figure: Activity life cycle

Scenario	Methods called
First launch	onCreate, onStart, onResume
Home button	onPause, onStop

Scenario	Methods called
Return to app	onRestart, onStart, onResume
Back button	onPause, onStop, onDestroy

Conclusion: Understanding the life cycle helps developers save data and release resources at the right time.

Q4. Write a program to start a second activity and pass data to it. How are activities declared? VERY LIKELY

Definition / Introduction: Activities communicate through Intents. An explicit intent names the target activity, and extras carry data as key-value pairs.

Points

- Create an **Intent** with the current context and the target class, add data with `putExtra()`, and call `startActivity()`.
- In the target activity, read the data with `getIntent().getStringExtra("key")`.
- Every activity must be **declared in the manifest** with an <activity> element, otherwise the app crashes with ActivityNotFoundException.
- `finish()` closes the current activity and returns to the previous one.

Passing data between two activities

```
// MainActivity.java
Button btn = findViewById(R.id.btnNext);
btn.setOnClickListener(v -> {
    Intent intent = new Intent(MainActivity.this, SecondActivity.class);
    intent.putExtra("name", "Amrit");
    startActivity(intent);
});

// SecondActivity.java (inside onCreate)
String name = getIntent().getStringExtra("name");
TextView tv = findViewById(R.id.tvName);
tv.setText("Hello " + name);

// AndroidManifest.xml
// <activity android:name=".SecondActivity" android:exported="false" />
```

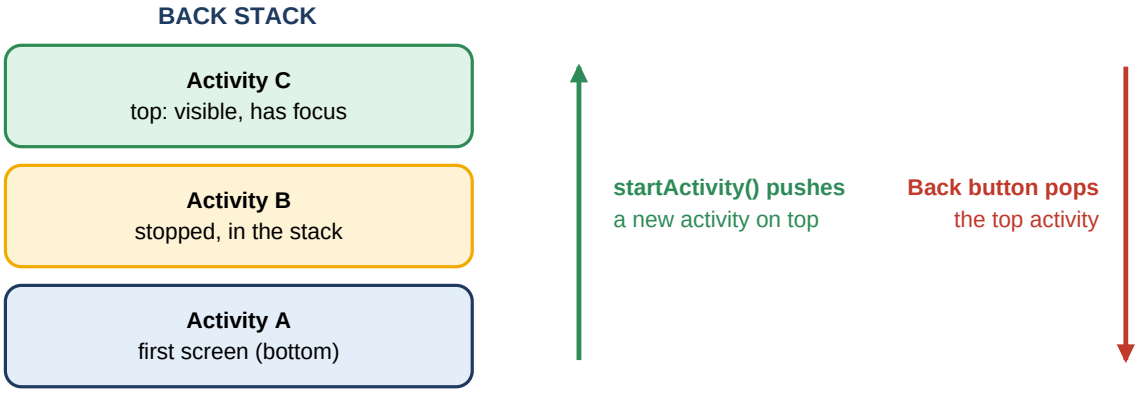


Figure: Back stack

Conclusion: An explicit intent with extras is the standard way of moving between screens and sharing data.

Q5. What is a Service? Explain its types and life cycle. VERY LIKELY

Definition / Introduction: A Service is an application component that performs long-running operations in the background and does not provide a user interface.

Points

- **Foreground service:** does work the user notices and must show a notification (for example a music player).
- **Background service:** work the user does not notice; heavily restricted since Android 8.0, so WorkManager is preferred.
- **Bound service:** a component binds with bindService(); it works like a client-server and runs only while clients are bound.
- **Started service life cycle:** startService() leads to onCreate(), then onStartCommand(), the service runs, and stopSelf() or stopService() leads to onDestroy().
- **Bound service life cycle:** bindService() leads to onCreate(), then onBind(); when all clients unbind, onUnbind() and onDestroy() are called.
- A service runs on the main thread by default; do heavy work on a background thread to avoid ANR. It must be declared in the manifest.

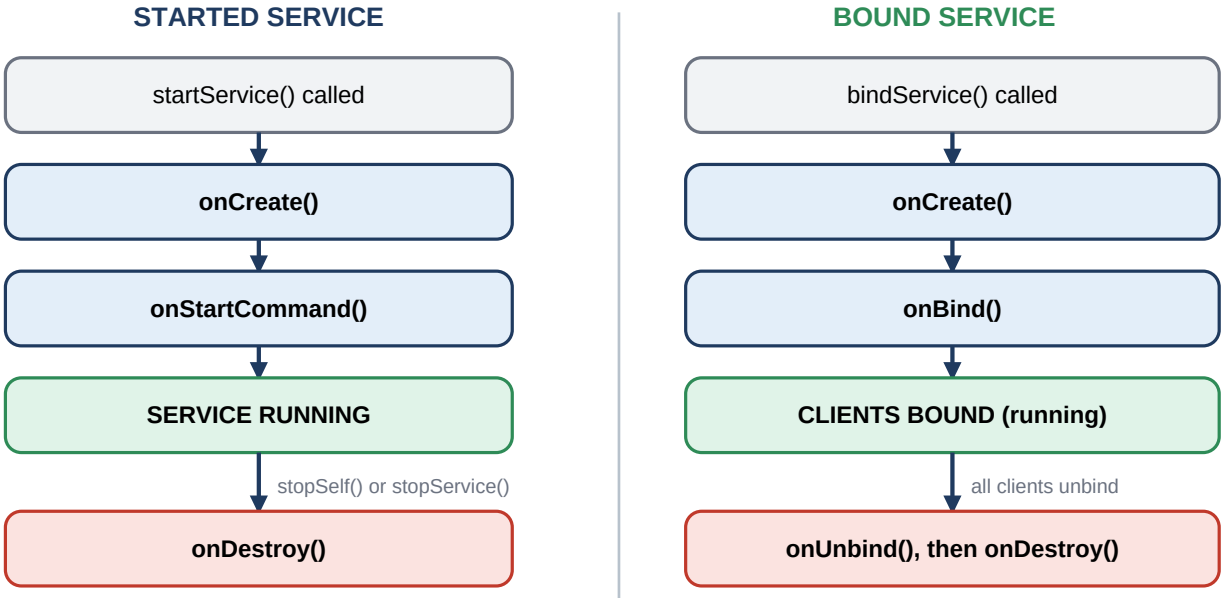


Figure: Life cycle of started and bound services

Conclusion: Services let an app do work that continues when the user leaves the screen.

Q6. Write a program to create and use a service (music player example). LIKELY

Definition / Introduction: A started service can play music in the background even when the user leaves the activity. We create a service class, declare it in the manifest and start and stop it from an activity.

Points

- **Step 1:** extend the `Service` class and override onCreate(), onStartCommand(), onBind() and onDestroy().
- **Step 2:** put the audio file in `res/raw` and create a MediaPlayer.
- **Step 3:** declare the service in the manifest.
- **Step 4:** start it with startService() and stop it with stopService() from button clicks.

MusicService.java and usage

```
public class MusicService extends Service {
    private MediaPlayer player;

    @Override public void onCreate() {
        super.onCreate();
        player = MediaPlayer.create(this, R.raw.song);
        player.setLooping(true);
    }
    @Override public int onStartCommand(Intent intent, int flags, int startId) {
        player.start();
        return START_STICKY;
    }
    @Override public IBinder onBind(Intent intent) { return null; }
    @Override public void onDestroy() {
        super.onDestroy();
        player.stop();
        player.release();
    }
}

// Activity: start / stop
startService(new Intent(this, MusicService.class));
stopService(new Intent(this, MusicService.class));

// Manifest:  <service android:name=".MusicService" android:exported="false" />
```

Conclusion: The service keeps playing the song until stopService() is called, independent of the screen.

Q7. What are Intents? Explain explicit and implicit intents with examples. VERY LIKELY

Definition / Introduction: An Intent is a messaging object used to request an action from another application component. It is used to start activities and services and to deliver broadcasts.

Points

- **Explicit intent:** names the exact target component class. Used inside your own app. Example: `new Intent(this, SecondActivity.class)`.
- **Implicit intent:** describes an action and data; the system finds a suitable app using intent filters. Example: `new Intent(Intent.ACTION_VIEW, Uri.parse("https://..."))`.
- **Parts of an intent:** component name, action, data, category, extras and flags.
- Data is passed with `putExtra()` and read with `getStringExtra()`; results come back via the Activity Result API (or the older startActivityForResult()).

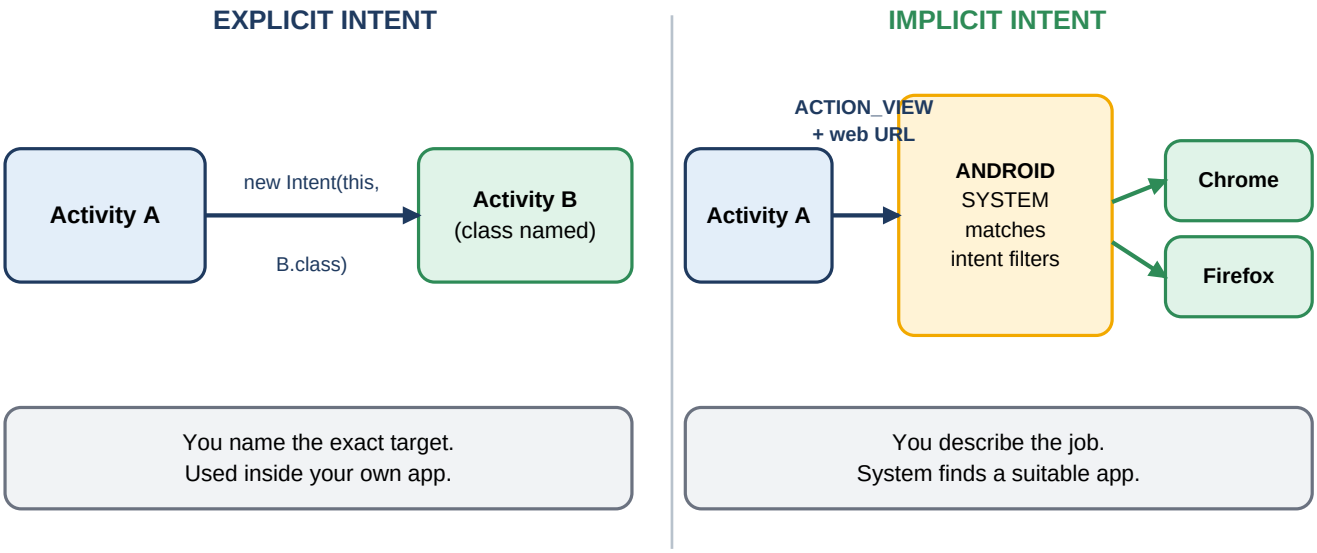


Figure: Explicit and implicit intents

Implicit intent examples

```
Intent dial = new Intent(Intent.ACTION_DIAL, Uri.parse("tel:100"));
startActivity(dial);

Intent send = new Intent(Intent.ACTION_SEND);
send.setType("text/plain");
send.putExtra(Intent.EXTRA_TEXT, "Hello");
startActivity(Intent.createChooser(send, "Share via"));
```

Conclusion: Intents make Android components loosely coupled and let apps reuse each other's features.

Q8. Explain how to receive and broadcast intents using a Broadcast Receiver.

VERY LIKELY

Definition / Introduction: A Broadcast Receiver is a component that listens for broadcast intents sent by the system or by apps and reacts to them. A broadcast is a message delivered to all interested receivers.

Points

- **Create a receiver:** extend `BroadcastReceiver` and override `onReceive(Context, Intent)`.
- **Register statically** in the manifest with <receiver> and <intent-filter>; it works even when the app is not running. Since Android 8.0 most implicit broadcasts cannot be declared this way (BOOT_COMPLETED is an exception).
- **Register dynamically** in code with registerReceiver() and an IntentFilter; unregister in onStop()/onDestroy(). On Android 14 specify RECEIVER_EXPORTED or RECEIVER_NOT_EXPORTED.
- **Send a broadcast:** create an Intent with a custom action and call `sendBroadcast()` (all receivers) or `sendOrderedBroadcast()` (one at a time by priority).
- **System broadcasts:** BOOT_COMPLETED, BATTERY_LOW, POWER_CONNECTED, AIRPLANE_MODE_CHANGED, SCREEN_ON/OFF.

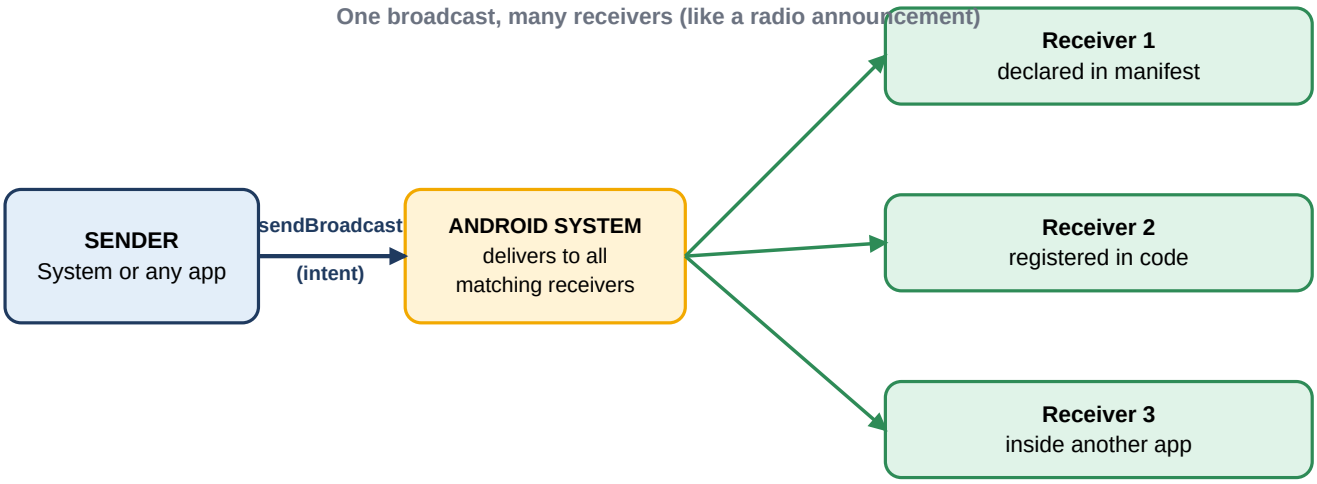


Figure: Broadcast to many receivers

Receiver and broadcast

```
public class MyReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        Toast.makeText(context, "Received: " + intent.getAction(),
                       Toast.LENGTH_SHORT).show();
    }
}

// Send a broadcast from an activity
Intent i = new Intent("com.example.myapp.MY_ACTION");
i.setPackage(getPackageName());
sendBroadcast(i);
```

Conclusion: Broadcasts let components react to events without knowing who sent them.

Q9. Explain the Android manifest file and its common settings. VERY LIKELY

Definition / Introduction: AndroidManifest.xml is a mandatory XML file in the root of every app that describes the app to the Android system: its package, components, permissions and requirements. The system reads it before running any code.

Points

- **<uses-permission>:** permissions the app needs (INTERNET, CAMERA). **<uses-feature>:** hardware needed, with android:required.
- **<application>:** icon, label, theme, allowBackup, name; it contains all components.
- **<activity>:** name, label, icon, theme, exported, launchMode, screenOrientation, parentActivityName.
- **<service>, <receiver>, <provider>:** declare a service, broadcast receiver and content provider (name, exported, permission, authorities).
- **<intent-filter>:** action, category and data; MAIN + LAUNCHER marks the launcher activity.
- **android:exported** must be set for components with intent filters when targeting Android 12 or higher.

Manifest example

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myapp">
    <uses-permission android:name="android.permission.INTERNET" />
    <application android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name" android:theme="@style/Theme.MyApp">
        <activity android:name=".MainActivity" android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <service android:name=".MusicService" android:exported="false" />
    </application>
</manifest>
```

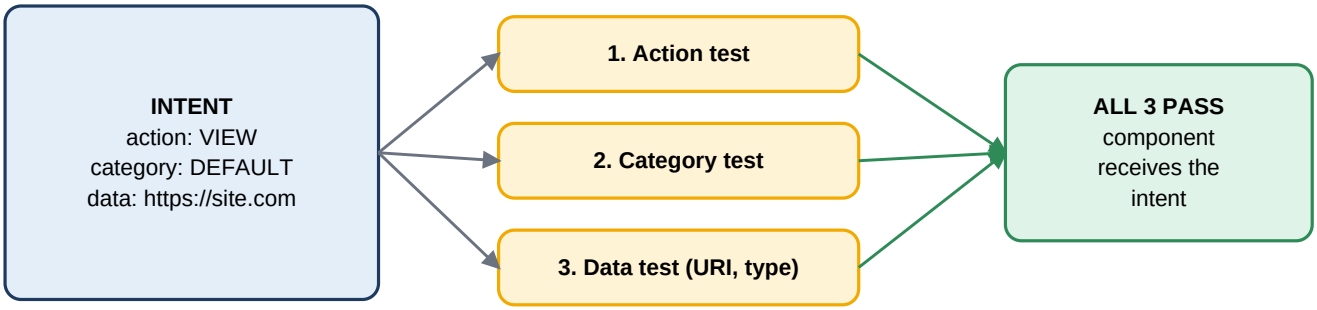
Conclusion: The manifest connects all parts of the app and tells the system what the app is and needs.

Q10. What is an Intent Filter? Explain with examples. VERY LIKELY

Definition / Introduction: An intent filter is an expression in the manifest that declares the types of intents a component can receive. The system compares an implicit intent with the filters of all apps to find matching components.

Points

- **<action>:** the action that can be performed (VIEW, SEND, MAIN).
- **<category>:** how the component is used (DEFAULT, LAUNCHER, BROWSABLE).
- **<data>:** the data type: scheme, host, port, path, mimeType.
- **Matching:** the intent must pass the action test, the category test and the data test. If several apps match, a chooser dialog is shown.
- **Uses:** launcher activity (MAIN + LAUNCHER), opening links (VIEW + BROWSABLE), receiving shared text (SEND), and receivers for broadcasts.



The system compares the intent with each component's intent filter

Figure: Intent matching

Receiving shared text

```
<activity android:name=".ShareActivity" android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.SEND" />
        <category android:name="android.intent.category.DEFAULT" />
        <data android:mimeType="text/plain" />
    </intent-filter>
</activity>
```

Conclusion: Intent filters let apps offer their features to other apps without knowing them in advance.

Q11. What are Android permissions? Explain their types. — VERY LIKELY

Definition / Introduction: A permission is a restriction that protects sensitive data and features of the device. An app must declare the permissions it needs in the manifest, and for dangerous permissions it must also ask the user at run time.

Points

- **Normal permissions:** low risk; granted automatically at install. Examples: INTERNET, ACCESS_NETWORK_STATE, VIBRATE.
- **Dangerous (runtime) permissions:** access private data or sensitive features; the user must approve at run time (Android 6.0+). Examples: CAMERA, RECORD_AUDIO, READ_CONTACTS, ACCESS_FINE_LOCATION, SEND_SMS, POST_NOTIFICATIONS.
- **Signature permissions:** granted only to apps signed with the same certificate.
- **Special permissions:** granted from a special Settings screen, e.g. SYSTEM_ALERT_WINDOW.
- **Declaration:** `<uses-permission android:name="android.permission.CAMERA" />` in the manifest. Users can revoke dangerous permissions later in Settings.

Type	Granted	Examples
Normal	At install, automatically	INTERNET, VIBRATE
Dangerous	User approves at run time	CAMERA, READ_CONTACTS
Signature	Same signing certificate	Custom permission
Special	From Settings screen	SYSTEM_ALERT_WINDOW

Conclusion: The permission model gives users control over their private data and makes apps more trustworthy.

Q12. Explain how to request a dangerous permission at run time with code. — LIKELY

Definition / Introduction: From Android 6.0 (API 23), dangerous permissions must be requested while the app is running, not only declared in the manifest.

Points

- **Step 1:** declare the permission in the manifest with <uses-permission>.
- **Step 2:** check it with `ContextCompat.checkSelfPermission()`.
- **Step 3:** if not granted, optionally explain why (shouldShowRequestPermissionRationale) and call `ActivityCompat.requestPermissions()`.
- **Step 4:** handle the user's answer in `onRequestPermissionsResult()` (or in the Activity Result API callback).

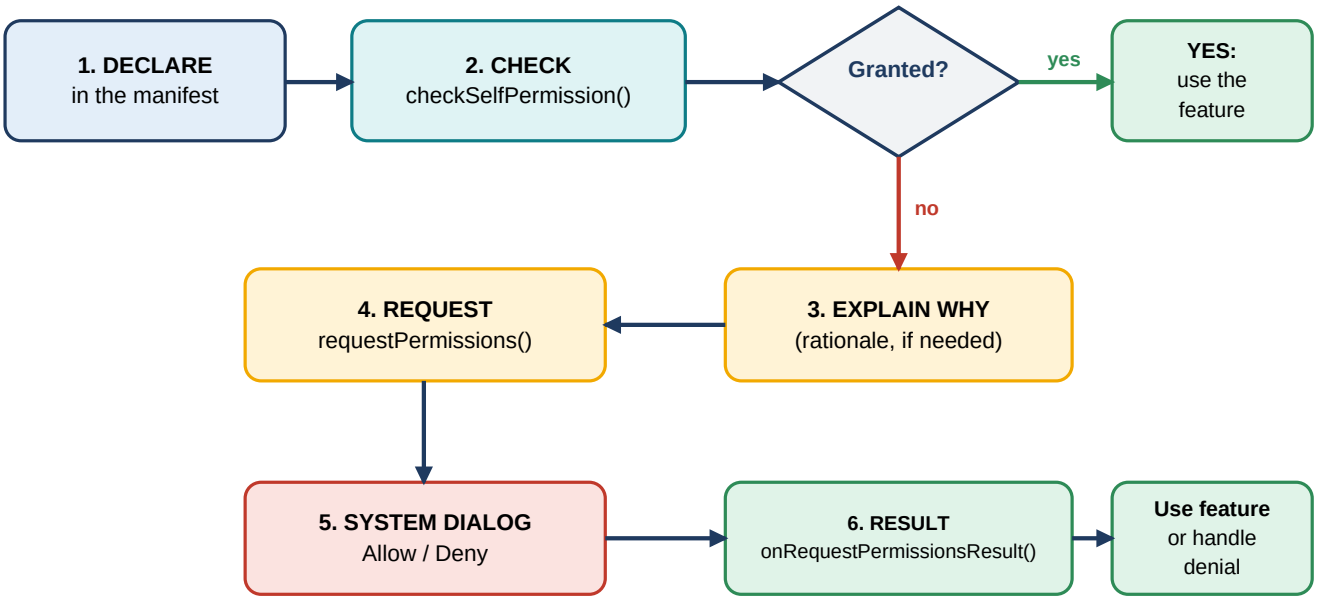


Figure: Runtime permission flow

Runtime permission code

```
if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA)
        != PackageManager.PERMISSION_GRANTED) {
    ActivityCompat.requestPermissions(this,
            new String[]{Manifest.permission.CAMERA}, 100);
} else {
    openCamera();
}

@Override
public void onRequestPermissionsResult(int code, @NonNull String[] perms,
                                       @NonNull int[] results) {
    super.onRequestPermissionsResult(code, perms, results);
    if (code == 100 && results.length > 0
            && results[0] == PackageManager.PERMISSION_GRANTED) {
        openCamera();
    }
}
```

Conclusion: Asking for a permission only when needed, with a clear reason, builds user trust.

Q13. Differentiate between: (a) Activity and Service, (b) Started and Bound service, (c) Explicit and Implicit intent.

POSSIBLE

Definition / Introduction: These pairs are often confused. The tables below compare them.

Points

- Each table compares the two items point by point.

(a) Point	Activity	Service
User interface	Has a UI (a screen)	No UI
Purpose	Interact with the user	Background work
Life cycle	onCreate, onStart, onResume, onPause, onStop, onDestroy	onCreate, onStartCommand / onBind, onDestroy
Example	Login screen	Music playing

(b) Point	Started service	Bound service
Started by	startService()	bindService()
Runs until	stopSelf() / stopService()	All clients unbind
Communication	None with the caller	Client-server through IBinder
Key callback	onStartCommand()	onBind()

(c) Point	Explicit intent	Implicit intent
Target	Class named	Action described
Used for	Inside own app	Using other apps
Resolved by	You	System via intent filters

Conclusion: Choosing the right component and intent type keeps the app simple and secure.

Quick Revision Sheet (read the night before)

Term	Remember this
4 components	Activity, Service, Broadcast Receiver, Content Provider
Context	Handle to app environment. getApplicationContext() vs this
Activity life cycle	onCreate, onStart, onResume | onPause, onStop, onRestart, onDestroy
Home / Back / Rotate	Home: onPause, onStop. Back: + onDestroy. Rotate: destroy then recreate
Save state	onSaveInstanceState(Bundle) and restore in onCreate()
Launch modes	standard, singleTop, singleTask, singleInstance
Service types	Foreground (notification), Background, Bound
Service callbacks	onCreate, onStartCommand, onBind, onUnbind, onDestroy
Start / stop service	startService() / stopService() or stopSelf()
Service rules	Android 8+ background limits. IntentService deprecated. Use explicit intents
Intent types	Explicit (class named) and Implicit (action + data)
Intent parts	Component, Action, Data, Category, Extras, Flags
Pass data	putExtra() and getIntent().getStringExtra()
Broadcast	sendBroadcast(), sendOrderedBroadcast(), onReceive()
Receiver registration	Manifest (static) or registerReceiver() (dynamic); unregister
Manifest settings	uses-permission, uses-feature, application, activity, service, receiver, provider
android:exported	Mandatory with intent-filter when targeting Android 12+
Intent filter	action + category + data; all three tests must pass
Launcher filter	MAIN + LAUNCHER
Permission types	Normal, Dangerous (runtime), Signature, Special
Runtime permission	Android 6.0+: checkSelfPermission, requestPermissions, onRequestPermissionsResult

Memory tricks

Activity life cycle: C-S-R-P-S-R-D = Create, Start, Resume, Pause, Stop, Restart, Destroy.
Visible = Start to Stop. Foreground = Resume to Pause. Entire = Create to Destroy.
Intent parts: C-A-D-C-E-F = Component, Action, Data, Category, Extras, Flags.
Permission types: N-D-S-S = Normal, Dangerous, Signature, Special.
Pause = Partly visible. Stop = Hidden.

Common Mistakes Students Make

- Mixing up **onPause()** (partly visible) and **onStop()** (not visible).
- Forgetting to **declare an activity or service in the manifest**. The app crashes.
- Doing heavy work in **onReceive()** or on a service's main thread, which causes ANR.
- Declaring only the permission in the manifest and **not requesting it at run time** for dangerous permissions.
- Not drawing the life-cycle diagram. It carries most of the marks.

Self-Test (answer without looking, 5 marks each)

- Draw and explain the Activity life cycle. What is called when the Home button is pressed?
- What is a Service? Explain started and bound services.
- Write a program to start a second activity and pass a name to it.
- What is an Intent Filter? Write the filter for the launcher activity.
- Explain the types of Android permissions. How is a dangerous permission requested?

Self-Test Answers (check after writing)

1. Draw Figure 3. Methods: onCreate, onStart, onResume, onPause, onStop, onRestart, onDestroy. Home button: **onPause(), then onStop()**. Returning: onRestart, onStart, onResume.

2. A service does long-running background work without a UI. Started: startService(), runs until stopSelf()/stopService(). Bound: bindService(), client-server through IBinder, stops when all clients unbind. Draw Figure 5.

3. Intent intent = new Intent(MainActivity.this, SecondActivity.class); intent.putExtra("name", "Amrit"); startActivity(intent); In SecondActivity: getIntent().getStringExtra("name"). Declare SecondActivity in the manifest.

4. An intent filter declares which intents a component can receive. Launcher: <intent-filter> <action android:name="android.intent.action.MAIN" /> <category android:name="android.intent.category.LAUNCHER" /> </intent-filter>.

5. Normal (auto at install), Dangerous (runtime), Signature (same certificate), Special (Settings screen). Dangerous: declare in manifest, check with checkSelfPermission(), request with requestPermissions(), handle onRequestPermissionsResult().

Done with Unit II!

If you scored 4 out of 5 or more, move to **Unit III**. If not, re-read the topics you missed in Part A.